

# Angular — User Login UI for SecurityService

This document contains a complete Angular login UI implementation that connects to your SecurityService login endpoint at:

```
http://127.0.0.1:8000/security-service/user/login
```

It includes components, services, guards, interceptor, models, routing, and usage instructions. Copy the files into an Angular project (Angular 15+ recommended) or create a new project and drop these files into `src/app`.

---

## Project structure (files provided below)

```
src/  
  app/  
    auth/  
      auth.service.ts  
      token.model.ts  
      auth.guard.ts  
      token.interceptor.ts  
      login/  
        login.component.ts  
        login.component.html  
        login.component.css  
      home/  
        home.component.ts  
        home.component.html  
        home.component.css  
      app-routing.module.ts  
      app.module.ts  
    environments/  
      environment.ts  
      environment.prod.ts  
  styles.css  
  README.md
```

### 1) src/environments/environment.ts

```
export const environment = {  
  production: false,  
  apiUrl: 'http://127.0.0.1:8000/security-service'  
};
```

### 2) src/environments/environment.prod.ts

```
export const environment = {  
  production: true,  
  apiUrl: 'http://127.0.0.1:8000/security-service'  
};
```

### 3) src/app/auth/token.model.ts

```
export interface TokenResponse {  
  access_token: string;  
  token_type: string;  
}
```

### 4) src/app/auth/auth.service.ts

```
import { Injectable } from '@angular/core';  
import { HttpClient, HttpResponse } from '@angular/common/http';  
import { Observable, throwError } from 'rxjs';  
import { catchError, map } from 'rxjs/operators';  
import { TokenResponse } from '../token.model';  
import { environment } from '../../environments/environment';  
  
@Injectable({ providedIn: 'root' })  
export class AuthService {  
  private baseUrl = environment.apiUrl;  
  private loginUrl = `${this.baseUrl}/user/login`;  
  
  constructor(private http: HttpClient) {}  
  
  login(email: string, password: string): Observable<TokenResponse> {  
    return this.http.post<TokenResponse>(this.loginUrl, { email,
```

```

password }).pipe(
    catchError(this.handleError)
);
}

setToken(token: string) {
    localStorage.setItem('access_token', token);
}

getToken(): string | null {
    return localStorage.getItem('access_token');
}

logout() {
    localStorage.removeItem('access_token');
}

isAuthenticated(): boolean {
    return !!this.getToken();
}

private handleError(error: HttpErrorResponse) {
    let message = 'An unknown error occurred';
    if (error.error && error.error.detail) {
        message = error.error.detail;
    } else if (error.status === 0) {
        message = 'Unable to reach server';
    } else if (error.status === 401) {
        message = 'Invalid credentials';
    } else if (error.status === 403) {
        message = 'Access forbidden';
    } else if (error.status === 404) {
        message = 'Resource not found';
    }
    return throwError(() => ({ status: error.status, message }));
}
}

```

## 5) src/app/auth/token.interceptor.ts

```

import { Injectable } from '@angular/core';
import { HttpInterceptor, HttpRequest, HttpHandler, HttpEvent } from '@angular/
common/http';
import { Observable } from 'rxjs';

```

```

import { AuthService } from './auth.service';

@Injectable()
export class TokenInterceptor implements HttpInterceptor {
  constructor(private auth: AuthService) {}

  intercept(request: HttpRequest<any>, next: HttpHandler):
  Observable<HttpEvent<any>> {
    const token = this.auth.getToken();
    if (token) {
      const cloned = request.clone({
        setHeaders: {
          Authorization: `Bearer ${token}`
        }
      });
      return next.handle(cloned);
    }
    return next.handle(request);
  }
}

```

## 6) src/app/auth/auth.guard.ts

```

import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';
import { AuthService } from './auth.service';

@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}

  canActivate(): boolean {
    if (this.auth.isAuthenticated()) {
      return true;
    }
    this.router.navigate(['/login']);
    return false;
  }
}

```

## 7) src/app/login/login.component.ts

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { AuthService } from '../auth/auth.service';

@Component({
  selector: 'app-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
export class LoginComponent {
  email = '';
  password = '';
  loading = false;
  error: string | null = null;

  constructor(private auth: AuthService, private router: Router) {}

  submit() {
    this.loading = true;
    this.error = null;
    this.auth.login(this.email, this.password).subscribe({
      next: (res) => {
        this.auth.setToken(res.access_token);
        this.loading = false;
        this.router.navigate(['/home']);
      },
      error: (err) => {
        this.loading = false;
        this.error = err?.message || 'Login failed';
      }
    });
  }
}
```

## 8) src/app/login/login.component.html

```
<div class="login-card">
  <h2>Sign in</h2>
  <form [(ngSubmit)="submit()" autocomplete="on">
    <label for="email">Email</label>
    <input id="email" name="email" type="email" [(ngModel)]="email" required />
```

```

    <label for="password">Password</label>
    <input id="password" name="password" type="password" [(ngModel)]="password"
required minlength="6" />

    <button type="submit" [disabled]="loading">{{ loading ? 'Signing in...' :
'Sign in' }}</button>

    <div class="error" *ngIf="error">{{ error }}</div>
  </form>
</div>

```

## 9) src/app/login/login.component.css

```

.login-card {
  width: 320px;
  margin: 48px auto;
  padding: 24px;
  border-radius: 8px;
  box-shadow: 0 6px 18px rgba(0,0,0,0.08);
}
.login-card h2 { text-align:center; margin-bottom: 12px; }
.login-card label { display:block; margin-top: 10px; font-weight:600; }
.login-card input { width:100%; padding:8px; margin-top:6px; box-sizing:border-
box; }
.login-card button { width:100%; padding:10px; margin-top:16px; border:none;
border-radius:6px; cursor:pointer; }
.error { color: #b00020; margin-top:10px; }

```

## 10) src/app/home/home.component.ts

```

import { Component } from '@angular/core';
import { AuthService } from '../auth/auth.service';
import { Router } from '@angular/router';

@Component({
  selector: 'app-home',
  templateUrl: './home.component.html',
  styleUrls: ['./home.component.css']
})
export class HomeComponent {

```

```
constructor(private auth: AuthService, private router: Router) {}

logout() {
  this.auth.logout();
  this.router.navigate(['/login']);
}
}
```

## 11) src/app/home/home.component.html

```
<div class="home">
  <h1>Welcome</h1>
  <p>You are signed in.</p>
  <button [(click)="logout()">Sign out</button>
</div>
```

## 12) src/app/app-routing.module.ts

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { LoginComponent } from '../login/login.component';
import { HomeComponent } from '../home/home.component';
import { AuthGuard } from '../auth/auth.guard';

const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: 'login', component: LoginComponent },
  { path: 'home', component: HomeComponent, canActivate: [AuthGuard] },
  { path: '**', redirectTo: 'home' }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

### 13) src/app/app.module.ts

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms';
import { HttpClientModule, HTTP_INTERCEPTORS } from '@angular/common/http';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { LoginComponent } from './login/login.component';
import { HomeComponent } from './home/home.component';
import { TokenInterceptor } from './auth/token.interceptor';

@NgModule({
  declarations: [
    AppComponent,
    LoginComponent,
    HomeComponent
  ],
  imports: [
    BrowserModule,
    FormsModule,
    HttpClientModule,
    AppRoutingModule
  ],
  providers: [
    { provide: HTTP_INTERCEPTORS, useClass: TokenInterceptor, multi: true }
  ],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

### 14) src/styles.css

```
body { font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
'Helvetica Neue', Arial; background: #f5f7fb; }
button { background: #1f6feb; color: #fff; }
```

## 15) README.md (usage)

```
# Angular User Login UI
```

1. Create a new Angular project (Angular CLI required):

```
```bash
ng new library-frontend --routing --style=css
cd library-frontend
```

1. Copy the files from this document into your project's `src/app` and `src/environments`.
2. Update `environment.apiUrl` if your backend URL changes.
3. Install dependencies (if needed):

```
npm install
ng serve
```

1. Open `http://localhost:4200` in your browser. Use credentials stored in your backend to sign in.

Notes: - The app stores `access_token` in `localStorage` and attaches it as `Authorization: Bearer <token>`. - Adjust token expiration checks or decode the JWT in `AuthService` if you want to validate expiration client-side before calling protected routes. ""

---

### Final notes

- This UI expects the backend to return a JSON with `access_token` and `token_type` (exactly as your FastAPI `authenticate_user` returns `{"access_token": ..., "token_type": "bearer"}`).
- If your backend returns different keys, update `TokenResponse` and places where `res.access_token` is referenced.

If you want, I can: - Add a nicer UI using Angular Material (forms, snackbar for error messages), - Add JWT decode & expiry handling, or - Provide a downloadable ZIP of these files.

---

End of document.